

Melody-to-Piano: Transformer-Based Full Piano Accompaniment Generation from Simple Melody

Mohammad Taghizadeh (301624889), Sydney Widjaja (301605811)

Abstract

This project studies melody-conditioned symbolic music generation for automatic piano accompaniment prediction. Given a monophonic melody sequence, the goal is to generate a structured polyphonic piano accompaniment with harmonic compatibility, rhythmic support, and plausible left- and right-hand register organization. We formulate the task as a conditional sequence-to-sequence learning problem over event-based MIDI tokens.

We implement a complete end-to-end pipeline on the POP909 dataset, including symbolic preprocessing, phrase segmentation, rule-based baseline generation, Transformer encoder-decoder models, and structured symbolic evaluation. We compare a melody-conditioned Transformer and a chord-conditioned Transformer, then investigate training and decoding refinements. The final results show that the melody-only model achieves slightly higher note-level and onset-level F1 after output cleaning, while the chord-conditioned model achieves better harmonic consistency and better register balance. On the full 1550-example test set, the melody-only model reaches note F1 of 0.0759 and onset F1 of 0.1995, whereas the chord-conditioned model achieves a higher chord-tone ratio of 0.6946 and lower register balance error of 0.1192. We further find that lightweight symbolic post-processing provides larger gains than architecture scaling or beam search, indicating that local decoding errors are a major bottleneck in this task.

1 Introduction

Automatic accompaniment generation from melody is a structured symbolic prediction problem. Unlike unconditional music generation, accompaniment prediction must align with the input sequence while producing musically plausible output. In the piano setting, this requires simultaneous modeling of harmonic support,

temporal alignment, chord density, and register allocation across left- and right-hand ranges.

This project studies melody-to-piano generation as a conditional sequence modeling task. In this approach, a melody token sequence serves as the source signal and a polyphonic accompaniment sequence serves as the target output. The primary goal is to determine whether attention-based sequence models can effectively learn accompaniment structure directly from symbolic datasets and whether explicit harmonic conditioning enhances the quality of the generated music.

This problem is interesting for two main reasons. First, it provides a controlled setting to study structured generation with weak alignment between input and output. The melody constrains on the harmonic and rhythmic possibilities, yet the accompaniment remains multimodal, allowing for multiple valid accompaniments for the same melody. Second, melody-conditioned accompaniment is practically beneficial in composition tools, symbolic arrangement systems, and educational software, especially since lead melodies are often more accessible than full piano realizations.

The key research questions are : Can a Transformer-based sequence-to-sequence model generate harmonically coherent and rhythmically structured piano accompaniment from melody input? Does explicit chord conditioning improve performance? And which components of the generation pipeline are most responsible for the final quality of the generated accompaniment?

2 Related Work

Symbolic music generation has traditionally been studied using recurrent neural networks and autoregressive sequence models. Early research in neural music modeling utilized recurrent architectures and LSTMs (Long Short-Term Memory) to represent temporal dependencies in pitch and

rhythm sequences. While these models demonstrated the feasibility of neural symbolic generation, they faced limitation in representing long-range structures and hierarchical repetition patterns (Eck and Schmidhuber, 2002; Hadjeres et al., 2017).

Transformer-based architectures later became dominant because self-attention provides direct access to long contextual windows. Music Transformer introduced relative positional attention, showing that long symbolic music sequences can be modeled more effectively when the architecture explicitly captures relative timing relationships (Huang et al., 2019). This is particularly relevant for accompaniment generation, where harmonic contexts may span several measures.

Another substantial area of study involves multi-track symbolic generation and structured conditioning. MuseGAN explored generation of multiple coordinated symbolic tracks and highlighted the challenge of preserving inter-track coherence (Dong et al., 2018). Although our task is simpler than full multi-track generation, accompaniment modeling still requires the system to learn polyphonic texture conditioned on an external melodic signal.

The POP909 dataset provides aligned melody, accompaniment, beat, and chord annotations, specifically designed for symbolic arrangement research (Wang et al., 2020). This dataset is particularly suitable for accompaniment learning, as it exposes both melodic structure and explicit harmonic labels. In contrast to unconditional generation and continuation tasks, this project focuses on melody-conditioned accompaniment prediction and utilizes chord conditioning as an explicit structural guide.

For optimization, we train all neural models using Adam optimizer, which is standard for sequence models and has provided stable convergence across our experiments (Kingma and Ba, 2015).

3 Approach

3.1 Task Formulation

Let $M = (m_1, \dots, m_T)$ denote the melody token sequence and $P = (p_1, \dots, p_S)$ denote the accompaniment token sequence. The task is modeled as learning a conditional distribution

$$P_\theta(P \mid M),$$

where P_θ is implemented as a Transformer encoder-decoder model trained with token-level cross-entropy loss under teacher forcing.

3.2 Symbolic Representation

MIDI tracks are converted into event-based symbolic tokens. The representation includes note on-set, note offset, temporal shift, and velocity events. A phrase-level segmentation step is used to divide long songs into fixed-length examples of 4 bars in the final implementation. This reduces sequence length variability and makes batch training more stable.

Only two tracks are used for the main task formulation:

- **MELODY**: used as the input sequence
- **PIANO**: used as the accompaniment target

The **BRIDGE** track is excluded from the current formulation because it acts as an auxiliary melodic layer rather than the main accompaniment texture. Including it would make the target less well-defined and introduce an additional melodic signal into what is intended to be a melody-to-accompaniment mapping.

A key point clarified from the milestone feedback is how output tokens correspond to a concrete musical output. The model does not predict a single note per time step. Instead, it predicts a sequence of event tokens that can encode multiple overlapping notes. Polyphonic accompaniment is obtained by allowing several note-on events to occur before corresponding note-off events. During reconstruction, the generated event sequence is replayed in time order and deterministically converted back into a note list and then a MIDI file. Therefore, the model output directly corresponds to a concrete symbolic piano accompaniment and can contain multiple simultaneous notes.

3.3 Model Variants

Three systems are studied.

Rule-Based Baseline. A deterministic accompaniment generator uses chord labels to produce simple accompaniment patterns. In our implementation, the baseline places chord tones in lower and middle registers at metrically regular positions. The purpose of this baseline is not to achieve high realism, but to provide a reproducible symbolic reference that uses explicit harmonic information without neural learning.

Melody-Conditioned Transformer (Mel-Transformer). This system uses a Transformer encoder-decoder in which the encoder receives melody tokens only. The decoder predicts accompaniment tokens autoregressively.

Chord-Conditioned Transformer (Chord-Transformer). This variant augments the encoder input with chord tokens. Chord information is appended to the melody representation using a separator token to provide explicit harmonic context. The chord labels are taken directly from the POP909 annotations and aligned with each segment during preprocessing. The comparison between the two neural models isolates the effect of harmonic conditioning.

3.4 Implementation Scope

The main project-specific implementation work includes:

- MIDI inspection and track extraction for POP909
- symbolic tokenization and segmentation pipeline
- chord parsing and conditioning input construction
- rule-based accompaniment baseline
- training and decoding scripts for melody-conditioned and chord-conditioned models
- evaluation code for symbolic metrics
- output-cleaning rules for duplicate-note removal, pitch-range clamping, and chord-aware repair

Standard PyTorch Transformer modules are used for the neural architecture itself, while task-specific preprocessing, reconstruction, decoding, and evaluation logic were implemented specifically for this project.

4 Experimental Setup

4.1 Dataset Analysis

Experiments use the POP909 dataset, which contains 909 pop songs with aligned symbolic annotations. After symbolic conversion, the songs are split into training (80%), validation (10%), and test (10%) partitions.

Property	Melody	Piano
Avg. notes	~320	~980
Texture	mono	poly
Pitch span	narrow	wide
Role	input	target

Table 1: Observed symbolic characteristics after pre-processing. The accompaniment target is substantially denser and more polyphonic than the melody input.

The dataset exhibits a clear difference between melody and accompaniment tracks, as summarized in Table 1. Melody lines are sparse, largely monophonic, and concentrated in a narrower pitch band. In contrast, piano accompaniment tracks contain substantially denser note activity and span a wider pitch range, reflecting the presence of bass notes, inner voices, and chord tones.

These observations are important because they show that accompaniment prediction is not a trivial copying problem. The model must learn to generate a denser symbolic structure than the input melody while remaining harmonically compatible with it.

4.2 Training Setup

We trained all models in PyTorch. The milestone models used $d_{\text{model}} = 256$, 8 attention heads, 4 layers, feed-forward dimension 512, dropout 0.1, batch size 8, and early stopping based on validation loss. In the final experiments, we additionally explored lower learning rates, larger feed-forward dimensions, more attention heads, smaller batch sizes, accompaniment-only targets, and decoding modifications.

Final Training Configuration. The final model settings differ from the milestone configuration in several concrete ways. The batch size was reduced from 8 to 4 to improve gradient stability. The learning rate was decreased from 1×10^{-4} to 5×10^{-5} to allow more stable convergence during later training stages. The feed-forward dimension was increased from 512 to 1024 in selected experiments, although this did not yield consistent gains. We also experimented with increasing the number of attention heads from 8 to 12 and using accompaniment-only targets during training, where non-piano tokens were excluded from the decoder target sequence.

Despite these modifications, most training-side changes resulted in only minor or inconsistent improvements. In contrast, output-cleaning strategies applied after decoding produced substantially

larger gains across all evaluation metrics.

The best milestone validation losses were 1.0854 for the melody-conditioned model and 1.0770 for the chord-conditioned model. Several architecture changes, including deeper networks and wider embeddings, did not yield reliable downstream gains. The most promising training-side changes were smaller batch size and accompaniment-only decoding target variants, but even these improvements were limited relative to later output-cleaning gains.

The current experiments use greedy decoding during evaluation. Beam search was also tested, but it degraded performance and substantially increased decoding time.

4.3 Evaluation Metrics

Prediction quality is measured using symbolic event-level metrics.

Note Precision and Recall. Let N_{pred} denote the set of predicted notes and N_{ref} denote the set of reference notes. Precision and recall are defined as

$$P = \frac{|N_{\text{pred}} \cap N_{\text{ref}}|}{|N_{\text{pred}}|}, \quad R = \frac{|N_{\text{pred}} \cap N_{\text{ref}}|}{|N_{\text{ref}}|}.$$

The note-level F1 score is

$$F_1 = \frac{2PR}{P + R}.$$

Onset Precision, Recall, and F1. The same formulation is applied to onset events only. These metrics ignore note duration and therefore focus more directly on rhythmic placement.

Chord-Tone Ratio. Harmonic consistency is measured using chord-tone ratio. Let G denote the set of generated notes and let $C(t_g)$ denote the set of pitch classes belonging to the chord active at the onset time of note g .

$$CTR = \frac{1}{|G|} \sum_{g \in G} \mathbf{1}(\text{pitch}(g) \in C(t_g)).$$

Higher chord-tone ratio indicates stronger harmonic compatibility between generated notes and the reference chord progression.

Metric	Milestone		Final	
	Mel	Chord	Mel	Chord
Note Prec.	0.0613	0.0665	0.0689	0.0698
Note Recall	0.0791	0.0800	0.0897	0.0849
Note F1	0.0676	0.0712	0.0759	0.0748
Onset Prec.	0.1452	0.1598	0.1780	0.1808
Onset Recall	0.1934	0.2003	0.2423	0.2305
Onset F1	0.1618	0.1740	0.1995	0.1976
Chord-tone ratio	0.4772	0.5168	0.6684	0.6946
LH ratio err.	0.1333	0.1201	0.1242	0.1192
RH ratio err.	0.1333	0.1201	0.1242	0.1192

Table 2: Comparison between milestone and final results. The final system improves substantially across all metrics, with gains primarily driven by output post-processing.

Register Balance Error. The accompaniment target must distribute note activity across lower and higher pitch ranges in a way that resembles piano left-hand and right-hand usage. Register balance error is computed as the absolute deviation between predicted and reference note proportions in these two ranges. Lower values indicate better agreement with the target texture.

4.4 Comparison Setup

We compare the rule-based baseline, melody-conditioned Transformer, and chord-conditioned Transformer. In the final stage, we also compare raw decoding against output-cleaned decoding, since this became the strongest source of improvement. The final tables therefore emphasize both model conditioning and post-processing choices.

5 Results

5.1 Milestone Results

For reference, the milestone results (see Table 2, left columns) showed that chord conditioning improved all reported metrics relative to melody-only decoding.

5.2 Final Results

After additional training and decoding experiments, the strongest gains came from lightweight symbolic post-processing. The final full-test-set results are shown in Table 2 (right columns).

There are several important observations to note. As shown in Table 2, the final metrics are substantially better than the milestone metrics for both models, especially in note F1, onset F1, and chord-tone ratio. Second, incorporating chord conditioning still enhances harmonic

consistency in the final system, raising chord-tone ratio to 0.6946 and better register balance. Third, the melody-only model slightly outperforms the chord-conditioned model in terms of exact note and onset reconstruction after the final post-processing. This suggests that chord information improves tonal compatibility more reliably than exact symbolic matching.

6 Analysis

Generated MIDI outputs from both neural models show that the system learns non-trivial accompaniment structure. The outputs are not empty, not reduced to a single repeated pitch, and not confined to an unrealistic piano range. This confirms that the pipeline is functioning correctly and that the models generate structured symbolic content rather than collapsing.

The most important empirical finding is that output post-processing was more effective than architecture scaling. We tested larger feed-forward dimensions, deeper models, more attention heads, smaller batch sizes, alternative target formulations, and beam search. These changes produced either small improvements or no consistent gains. In contrast, lightweight symbolic cleanup—duplicate-note removal, pitch clamping, and conservative chord-aware snapping—improved all major metrics on validation and test examples.

This result suggests that the learned representations already contain useful musical structure, but the decoded outputs contain local symbolic errors that hurt note matching and harmonic consistency. In other words, the bottleneck is not only model capacity; it is also the quality of the raw decoded event sequence.

Another consistent pattern is that onset-level metrics remain much higher than note-level metrics. This indicates that temporal placement is easier for the model than exact pitch-duration realization. Failures are especially common in rhythmically dense passages, where the model often under-generates inner notes or fails to sustain chordal texture. The left-hand region also remains harder to model than the upper accompaniment range, although chord conditioning and output cleaning both reduce this gap.

Beam search did not help in our setting. It degraded note F1, onset F1, chord-tone ratio, and register balance while also increasing evaluation

time. This suggests that, for this task and model size, greedy decoding followed by lightweight symbolic repair is more effective than maintaining multiple candidate sequences during decoding.

7 Implementation and Work Allocation

The project code includes dataset preprocessing, vocabulary construction, model training, symbolic decoding, evaluation, and MIDI generation. Standard PyTorch Transformer modules were used for the neural architecture. All task-specific data handling, metric computation, decoding logic, and symbolic cleanup rules were implemented within the project codebase.

Work was divided as follows. Mohammad focused primarily on model implementation, training, debugging, evaluation, and analysis. Sydney contributed to dataset setup, baseline support, related work, and report preparation. Both authors contributed to the project design, interpretation of results, and final write-up.

8 Conclusion

This work investigated melody-conditioned piano accompaniment generation using Transformer-based sequence-to-sequence models on symbolic music data. The task was formulated as generating polyphonic accompaniment conditioned on a monophonic melody, with optional chord information as additional input. Experiments were conducted to evaluate the effect of harmonic conditioning, model scaling, decoding strategies, and output post-processing.

The final results show that both melody-only and chord-conditioned models are capable of producing structured accompaniment with non-trivial rhythmic and harmonic organization. Using greedy decoding combined with lightweight symbolic post-processing, the melody-only model achieved a note-level F1 score of 0.0759 and an onset-level F1 score of 0.1995. Incorporating chord information led to improved harmonic consistency, increasing the chord-tone ratio from 0.6684 to 0.6946, and slightly improving register balance error from 0.1242 to 0.1192. However, the effect of chord conditioning on note-level and onset-level F1 scores was limited, suggesting that while harmonic context improves pitch selection relative to chords, it does not substantially improve exact note reconstruction under the current modeling setup.

A key finding of this work is that output post-processing plays a significant role in improving generation quality. The raw model outputs contain local inconsistencies such as duplicate notes, off-chord pitches, and minor symbolic artifacts. A simple post-processing step that enforces basic constraints—removing duplicates, clamping pitch ranges, and snapping near-chord tones—consistently improves all evaluation metrics. This indicates that the learned representations capture useful musical structure, but the decoding process introduces errors that can be corrected with lightweight symbolic rules.

In contrast, increasing model capacity and applying beam search decoding did not yield improvements. Larger feed-forward dimensions, additional heads, and deeper configurations produced similar validation losses and did not translate into better evaluation metrics. Beam search further degraded performance, likely due to overconfidence in locally optimal token sequences that do not align with global musical structure. These results suggest that, for this task, improving output consistency is more effective than increasing model complexity.

Overall, the experiments demonstrate that melody-conditioned Transformer models can learn meaningful accompaniment structure from symbolic datasets, and that harmonic conditioning improves chord-level compatibility. However, precise polyphonic reconstruction remains challenging, as reflected in the persistent gap between onset-level and note-level metrics. Future work could explore alternative representations that better capture sustained chords and voice structure, as well as training objectives that directly optimize structural consistency rather than token-level likelihood.

References

- Hao-Wen Dong, Wen-Yi Hsiao, Li-Chia Yang, and Yi-Hsuan Yang. 2018. Musegan: Multi-track sequential generative adversarial networks for symbolic music generation and accompaniment. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Douglas Eck and Jurgen Schmidhuber. 2002. Finding temporal structure in music: Blues improvisation with lstm recurrent networks. In *Proceedings of the 12th IEEE Workshop on Neural Networks for Signal Processing*.
- Gaetan Hadjeres, Francois Pachet, and Frank Nielsen. 2017. Deepbach: A steerable model for bach chorales generation. In *Proceedings of the 34th International Conference on Machine Learning*.
- Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Ian Simon, Curtis Hawthorne, Noam Shazeer, Andrew M. Dai, Matthew D. Hoffman, Monica Dinculescu, and Douglas Eck. 2019. Music transformer: Generating music with long-term structure. In *International Conference on Learning Representations*.
- Diederik P. Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.
- Ziyu Wang, Ke Chen, Junyan Jiang, Yiyi Zhang, Mao-ran Xu, Shuqi Dai, Xianbin Gu, and Gus Xia. 2020. Pop909: A pop-song dataset for music arrangement generation. In *Proceedings of the 21st International Society for Music Information Retrieval Conference*.